CODEFLOW PSUT · SESSION: DESIGN PHASE

UI/UX Design: From Requirements to User Experience

Industry Practices Using the SDLC

Codeflow PSUT

November 4, 2026



DISCUSSION QUESTION · 01

What is the difference between UI and UX?

Talk to the person next to you

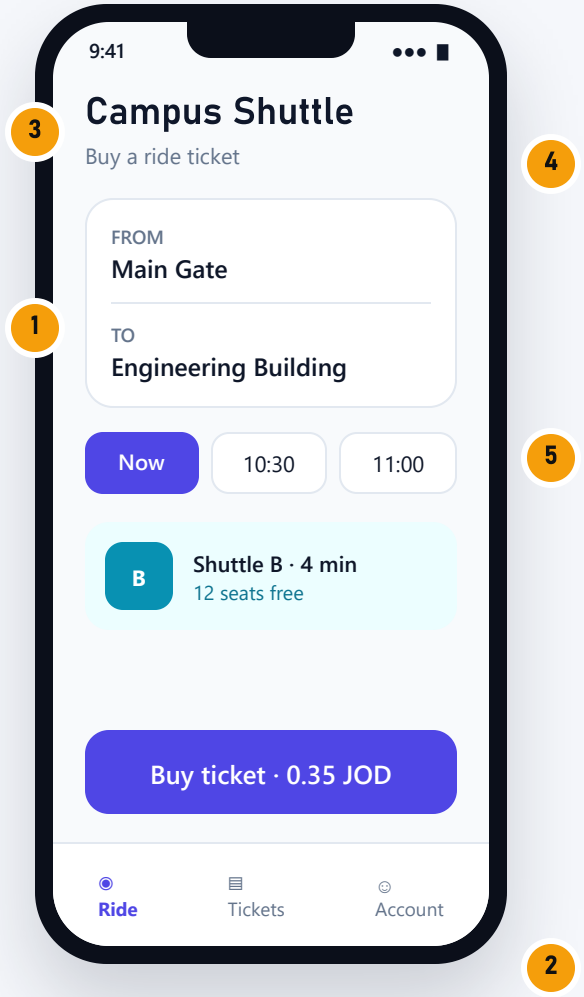
2 minutes

FOUNDATIONS

What is UI?

User Interface — everything a person sees, hears and touches to operate a product: the visible *layer* between the user and the system.

1. **Visual elements** — icons, images, cards, colour blocks
2. **Buttons, forms & navigation** — the controls people operate
3. **Typography & colour** — font, size, weight, palette, contrast
4. **Layout & visual hierarchy** — what the eye sees first, second, third
5. **Interactive elements** — toggles, tabs, states (pressed, disabled, loading)



Example: a campus shuttle app. Numbers match the UI elements on the left.

Think of UI as the "what": the screen-level decisions you could point at in a screenshot.

FOUNDATIONS

What is UX?

User Experience — how it feels to *reach a goal* with a product, from first intention to final outcome. It includes the screens, but also timing, wording, effort, errors and what happens off-screen.

Journey: “I need to get to my 10:30 lecture”

Same shuttle app · each step is part of the experience



User goals
Why are they here?

Ease of tasks
How much effort?

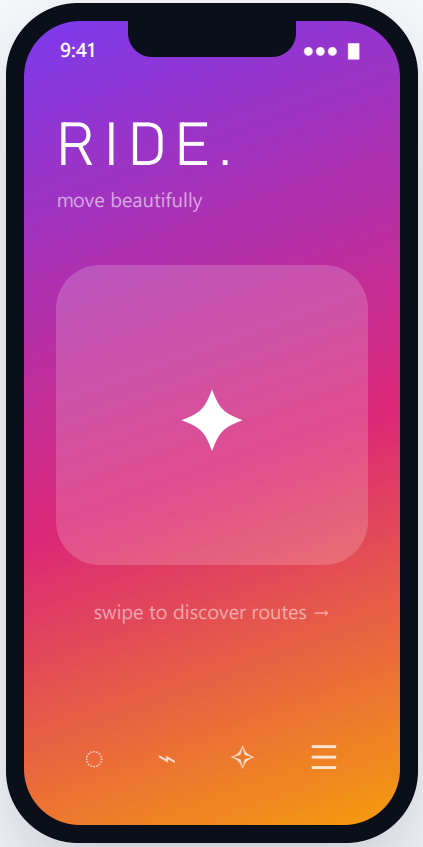
Navigation & flow
Is the next step obvious?

Efficiency & satisfaction
Fast? Pleasant?

Errors & recovery
Prevented? Fixable?

FOUNDATIONS

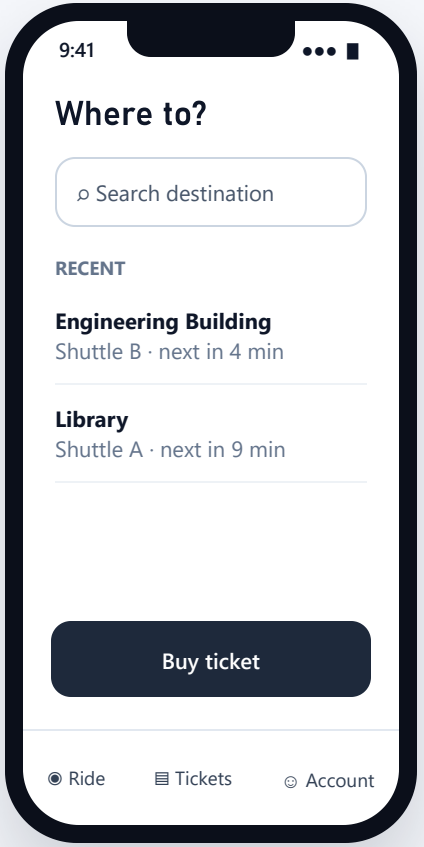
UI vs. UX: looking good ≠ working well



A · Stunning UI

- Unlabelled icons — guess the navigation
- Low-contrast text on gradients
- Routes hidden behind a swipe gesture
- 7 steps to buy one ticket

Beautiful screenshot.
Frustrating journey.



B · Plain UI, great UX

- Opens on the user's actual goal
- Remembers recent destinations
- Labelled navigation, high contrast
- 3 steps to buy one ticket

Ordinary screenshot.
Effortless journey.

UI = the interface's appearance and behaviour on each screen.

UX = the whole journey toward a goal — across screens, time and channels.

DISCUSSION QUESTION · 02

Why does UI/UX matter in software engineering?

Think beyond “it looks nicer”

2 minutes

IMPACT

Why UI/UX matters

Task completion
Can people actually finish what they came to do?

User satisfaction
Will they choose to come back?

Accessibility
Can people with disabilities use it at all?

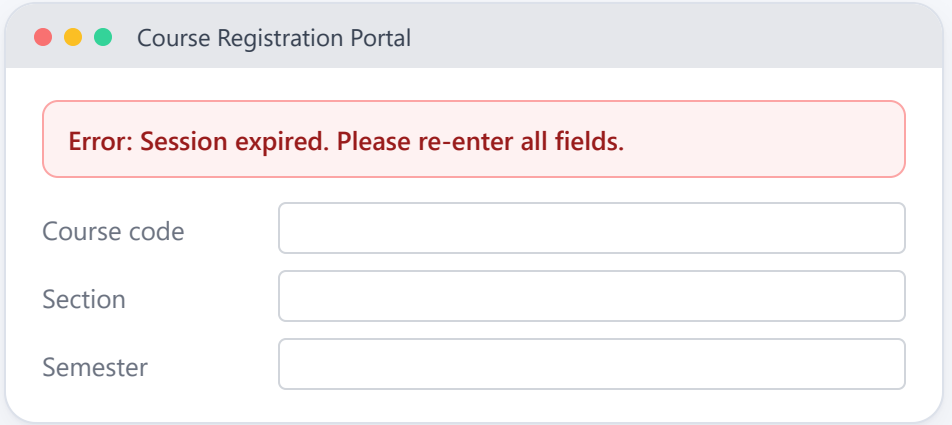
Error rates
Confusing UI creates wrong data and support tickets.

Development cost
A fix in a sketch takes minutes; after release it takes sprints.

Customer retention
Users leave products that waste their time.

Business success
Adoption, reputation and cost all depend on whether real users succeed.

Case · “Every test passes”



- Every function works and passes its tests ✓
- Sessions time out after 3 minutes; forms clear on error
- Result: students queue at the registrar’s office instead

“Functionally correct” is not the same as usable.

Illustrative scenario for discussion.

PRINCIPLES

What makes a good user experience?

1 Clarity

Use words users know. “Pay now”, not “Execute transaction”.

H2 · Match the real world

2 Consistency

Same action, same look, same place — on every screen.

H4 · Consistency & standards

3 Simplicity

Show what matters now; remove what doesn’t help the task.

H8 · Minimalist design

4 Accessibility

Readable contrast, labelled controls, keyboard & screen-reader support.

WCAG 2.2

5 Feedback

Every action gets a response: “Saving…”, “Saved ✓”, “Failed — retry”.

H1 · Visibility of status

6 Error prevention

Disable impossible choices; explain errors in plain language.

H5 + H9 · Prevent & recover

7 Efficiency

Fewest steps for frequent tasks: defaults, recents, autofill.

H7 · Flexibility & efficiency

8 User control

Back, cancel and undo are always available.

H3 · Control & freedom

“H” tags refer to Jakob Nielsen’s 10 usability heuristics (Nielsen Norman Group). WCAG = W3C Web Content Accessibility Guidelines.

DISCUSSION QUESTION · 03

Should every user be required to sign in before using a system?

Yes? No? It depends — on what?

3 minutes

THINKING LIKE A USER

Understanding user goals

A student wants to know one thing: **“Does the library have this book?”** Developers often design around what *the system* needs first.

System-first flow

Open → Sign in → Dashboard → Search → Answer

The answer arrives last. A guest, a forgotten password, or a slow login blocks a simple question.

Goal-first flow

Open → Search → Answer → Sign in only to reserve

The answer comes first. Identity is requested only when it becomes necessary.

Progressive disclosure
Reveal complexity (forms, options, sign-in) only when the user reaches the step that needs it.

Remove unnecessary steps
Every extra screen between the user and their goal is a chance to give up.

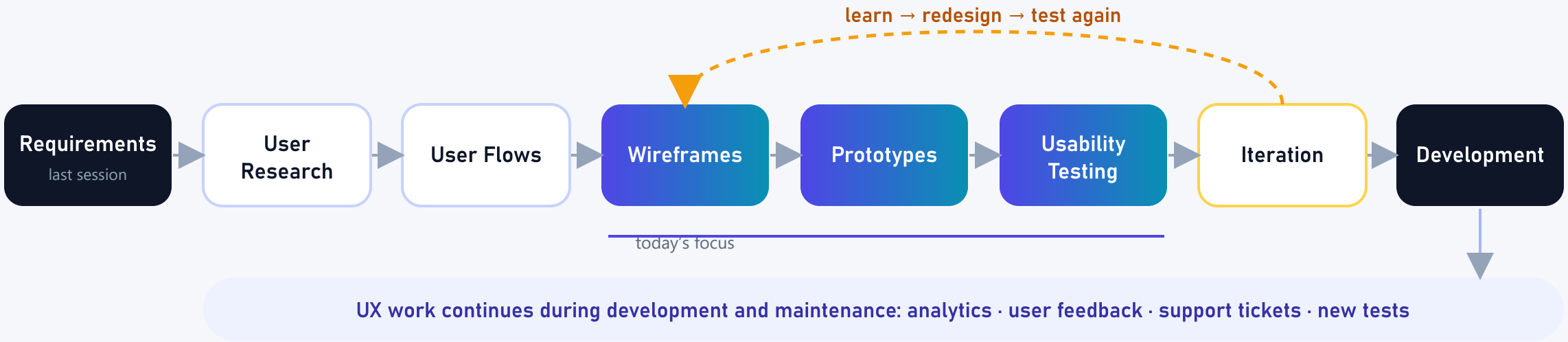
But: protect what must be protected
Authentication is still required before protected actions — reserving, viewing personal loans or fines.

ASK AT EVERY STEP “What is the **least** the user must give us **right now** — and why do we need it **here**?”

PROCESS

Where does UI/UX fit into the SDLC?

It starts where the last session ended — with requirements — and keeps looping.



Input
Requirements, stakeholders, constraints, business rules

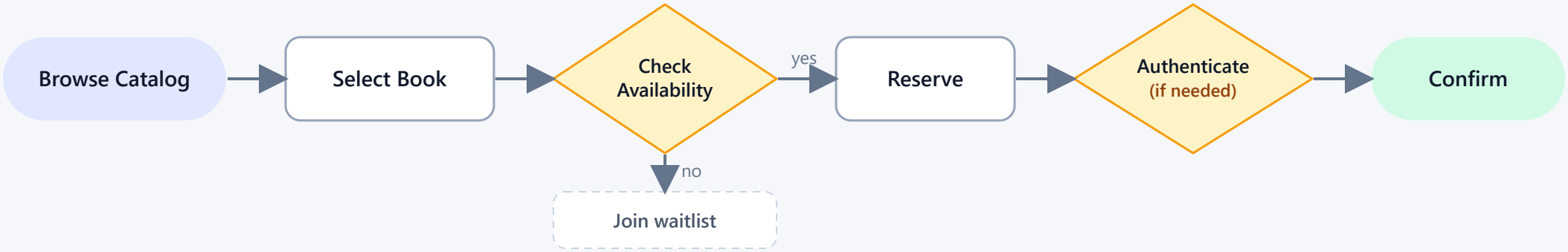
Output
Flows, wireframes, a tested prototype, design decisions with reasons

Reality check
Testing findings often send you back to the flows — or even the requirements

DESIGN ARTEFACTS

What is a user flow?

A diagram of the **steps and decisions a user goes through** to complete one goal. It helps teams spot missing screens, dead ends and unnecessary steps — before anything is drawn in detail.



User flow
What the **user does**, step by step, toward a goal. Screens + decisions.

Database schema
How **data is stored**: tables, keys and relationships. No user steps.

Architecture diagram
How **system components** connect: app, API, database, services.

DESIGN ARTEFACTS

What is a wireframe?

A simplified, usually grayscale blueprint of a screen. It shows **structure**, **layout**, **content hierarchy** and **navigation** — deliberately leaving out colour, imagery and polish.

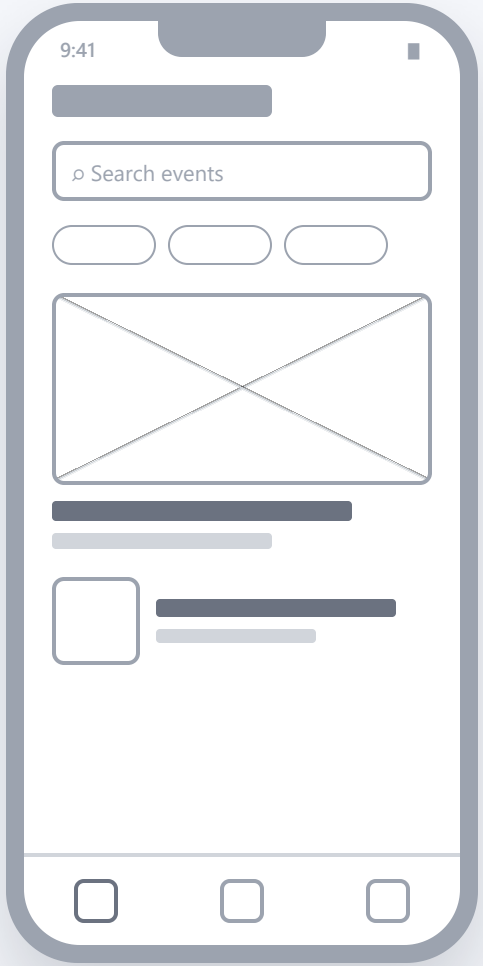
Why wireframe before development?

Cheap to change
Move a box, not a component tree.

Aligns the team
Devs, designers & clients see the same plan.

Exposes gaps
Missing requirements and states show up early.

Testable early
Users can react before code exists.



← Hierarchy
Title, then search: the most important things first

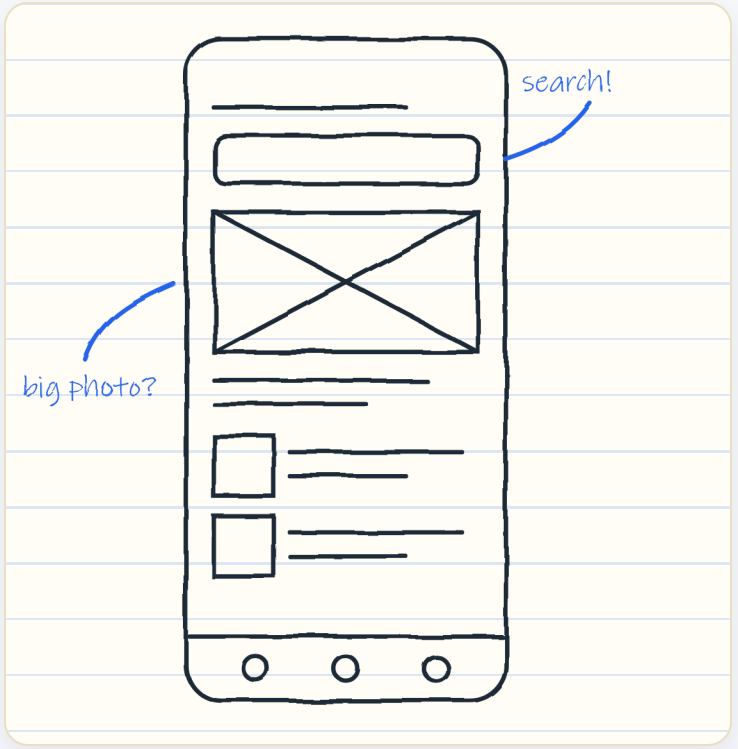
← Layout
Featured card, then list items

← Content
Grey bars = text, crossed box = image

← Navigation
Persistent bottom tab bar

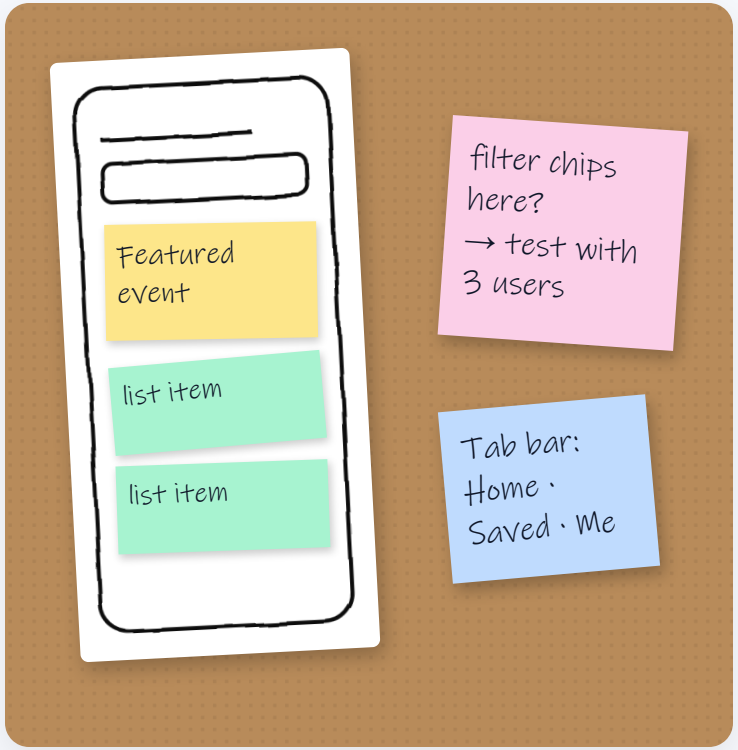
FIDELITY

Low-fidelity wireframes



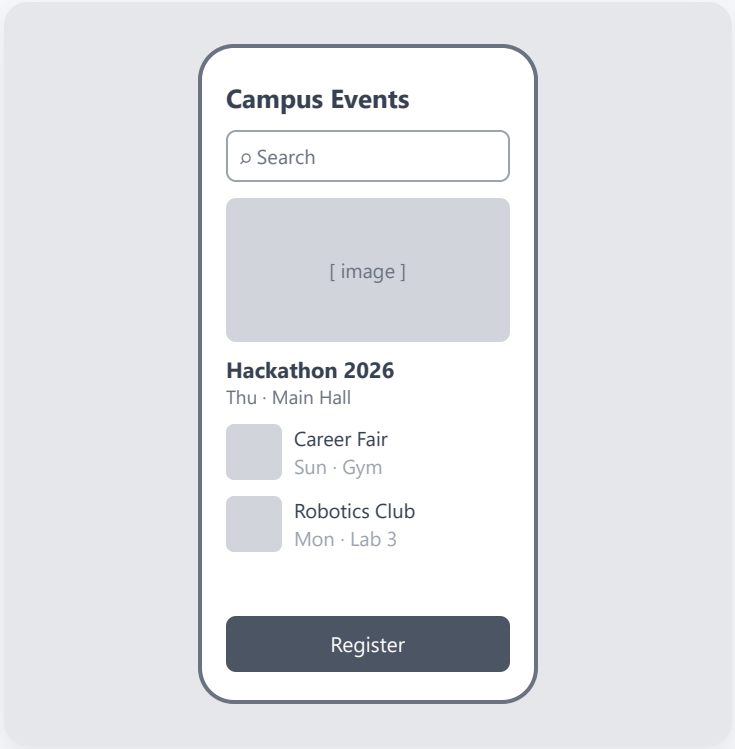
Hand-drawn sketch

Minutes per idea. Draw five, keep one.



Paper wireframe

Rearrange sticky notes with users in the room.



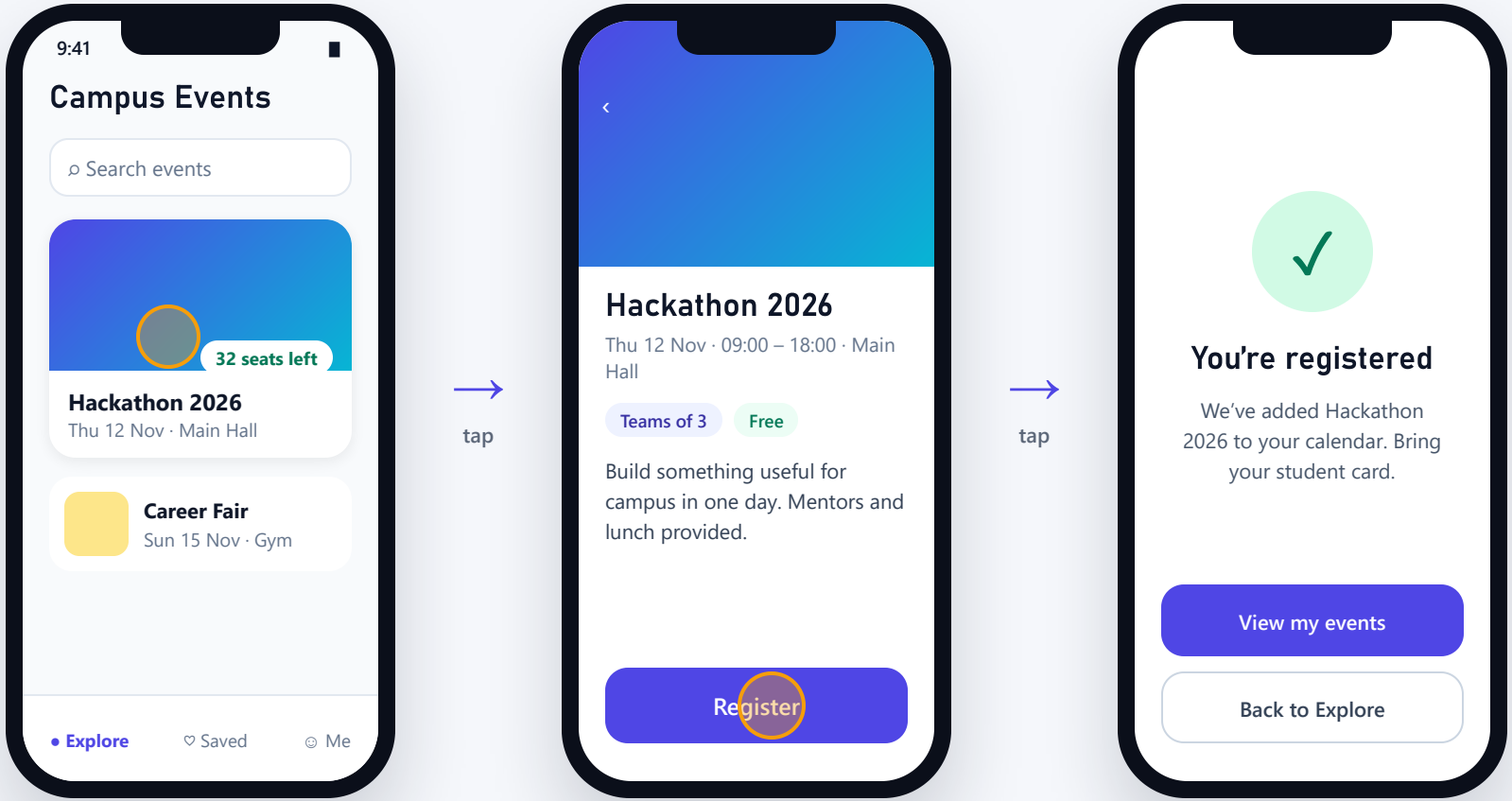
Grayscale digital

Clean boxes in Figma/FigJam — easy to share & link.

Why low-fi? Fast · Cheap · Easy to change · Encourages experimentation · Focus on structure, not styling

FIDELITY

High-fidelity prototypes



Typography — real fonts, sizes and weights

Colours — the final palette and contrast

Components — cards, chips, buttons, tab bar

Visual states — default · pressed · disabled

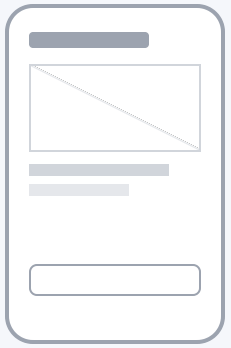
Interactive navigation — hotspots link screens

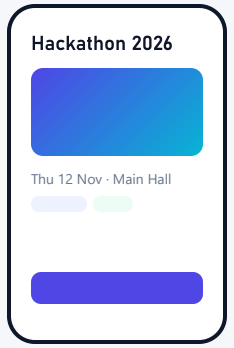
Realistic task flows — test a whole goal

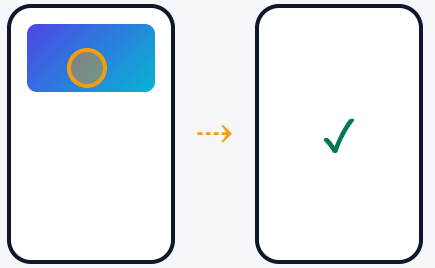
Careful: a polished **mockup** is just a picture. It becomes a **prototype** only when it simulates interaction.

FIDELITY

Wireframe vs. mockup vs. prototype







	Wireframe	Mockup	Prototype
PURPOSE	Agree on structure & flow	Agree on visual look	Test how it **works** with users
DETAIL	Low: boxes, grey, placeholders	High: real colours, type, images	Low *or* high — interaction is the point
INTERACTIVE?	No	No (static image)	**Yes** — clickable, linked screens
EFFORT	Minutes – hours	Hours – days	Hours – days
TYPICAL TOOLS	Paper, FigJam, Figma	Figma	Figma prototyping, paper prototypes

INDUSTRY TOOLKIT

Industry design tools — and where they fit

Explore · Map flows · Design screens · Stay consistent · Build faster · Validate

Pen & paper

Rapid sketching, crazy-8s, paper prototypes. No learning curve.

FigJam

Online whiteboard for user flows, journey maps and team workshops.

Figma

Collaborative wireframes, mockups, prototypes and developer hand-off.

Design systems

Shared rules: colours, type, spacing, patterns. e.g. Material Design, Apple HIG.

Component libraries

Reusable coded parts that match the design system: buttons, inputs, cards.

Usability testing

Moderated sessions, unmoderated tools (e.g. Maze), screen recording & surveys.

A typical team week

Sketch 6 ideas on paper → map the chosen flow in FigJam → wireframe in Figma using the team's design system → link a clickable prototype → test with 5 users → engineers build with the component library.

Product names are trademarks of their owners; logos intentionally omitted. Tools change — the workflow skills transfer.

MOBILE-FIRST

Why mobile-first?

Design for the **smallest, most constrained screen first**, then enhance for larger ones. Constraints force you to decide what truly matters.

Limited screen space
No room for “nice to have”.

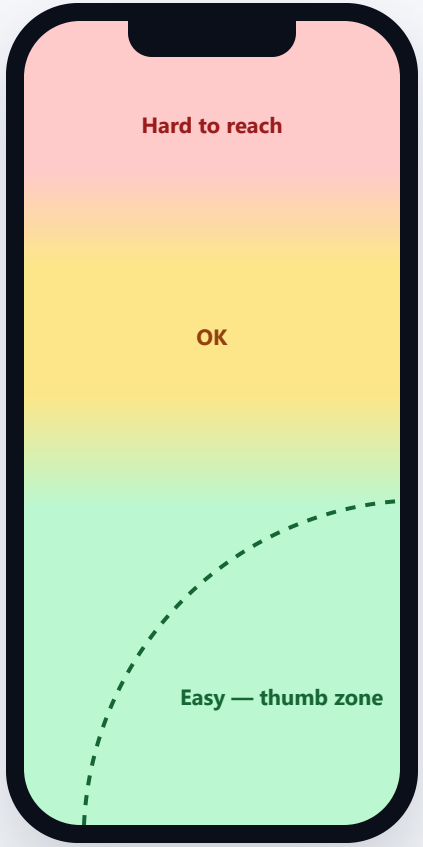
Content prioritisation
The primary task goes on top.

Touch interactions
Fingers, not precise mouse pointers.

Thumb-friendly navigation
Key actions within easy reach.

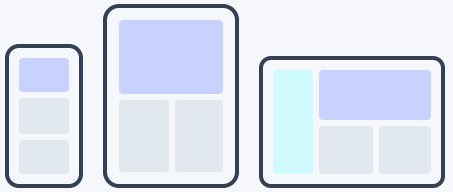
Responsive layouts
One design adapts to every width.

Progressive enhancement
Add columns & shortcuts as space grows.



One-handed use, right thumb

SAME CONTENT, MORE SPACE

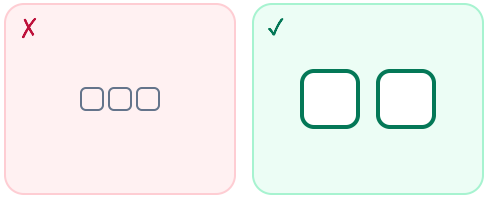


Case-study hypothesis: we *assume* most students will use the library system on their phones. This is not a verified statistic about PSUT students — a real team would confirm it with surveys or analytics.

MOBILE-FIRST

Mobile-first design principles

Accessible touch targets



WCAG 2.2 (2.5.8) sets a 24×24 px minimum; platform guides recommend ~44–48 px.

Readable typography



~16 px body text; text must still work when zoomed to 200% (WCAG 1.4.4).

Clear navigation



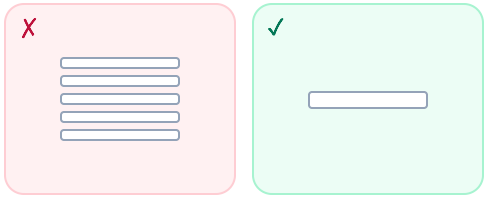
Icons with text labels; few, predictable destinations.

Search visibility



If search is the main task, put it at the top — not in a menu.

Minimal forms



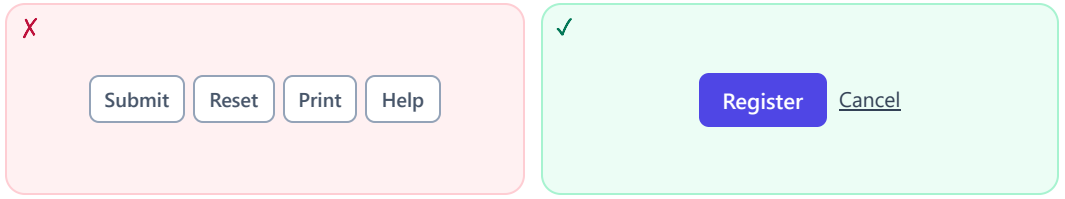
Ask only for what you can't already know. Prefill the rest.

Appropriate contrast



Text ≥ 4.5:1 (WCAG 1.4.3); buttons & icons ≥ 3:1 (1.4.11).

Clear primary action



One visually dominant action per screen, labelled with what it actually does.

CASE STUDY

University Library Management System

Students browse, search and reserve books online, then **collect and return them physically** at the library.
We'll trace how written requirements become screens.

G

Guest

Browses and searches the public catalogue.

S

Student

Reserves books, tracks loans, due dates and fines.

L

Librarian

Prepares reservations, confirms hand-overs and returns.

A

Administrator

Manages policies, staff permissions and configuration.

CORE JOURNEY Search online → Reserve online → Librarian prepares → Collect in person → Track due date → Return in person

📄 Handout: 02 — Requirements (SRS)

📱 Handout: 03 — Wireframes exercise

THE CHALLENGE

The system works. But is it usable?

You'll receive a version of the library app that meets the requirements on paper — but was designed without enough UX thinking. Your job: find what creates friction, then fix it.

HANDS-ON ACTIVITY

Redesign the Library System

⏱ 15–20 min

👥 Work in pairs

1 Study the bad interface

Walk through every screen in Part A as a student would.

2 Identify problems

Find the obvious ones — then look for the subtle ones.

3 Discuss the friction

Which design decisions slow users down or mislead them?

4 Sketch a better version

Homepage, catalogue or reservation flow — on paper.

5 Present & justify

Link each decision to a user need or a requirement.

6 Compare

Check your redesign against the instructor's reference.

You need
Handout 02 (SRS) · Handout 03, Parts A–B · pen & paper

You produce
A ranked list of ≥ 5 problems + one redesigned flow sketch

You present
2 minutes: the problem you fixed first, and why

GUIDE YOUR CRITIQUE

Discussion questions

1. What is the first thing a user wants to do?
2. What is making that task difficult?
3. Is the navigation understandable?
4. Which information is unnecessary?
5. Are users receiving enough feedback?
6. Does the interface meet accessibility expectations?
7. What happens when something goes wrong?

FINAL TAKEAWAY

Good UI/UX is not just about making software ~~look good~~.

01

Make the **correct actions easy**

02

Make the system's **behaviour understandable**

03

Make mistakes **preventable or recoverable**

Justify every design decision with **user needs**, **requirements** and **usability evidence** — not personal taste.